T-Route Workshop Guide

There are two ways to get a working T-Route, ngen (NextGen Framework), and ngen-cal (calibration tool) environment, depending on whether you have access to a Linux environment:

- **No Linux access?** Use the pre-built image. Install Docker Desktop and pull the image with T-Route, ngen, and ngen-cal already compiled. Go directly to **Part 1.**
- **Have Linux access?** Build from source code and follow **Part 2**.

Part 1: Run the Pre-Built Image (No Linux Required)

Workshop participants who do not have Linux access do not compile anything. The pre-built workshop image already contains compiled T-Route, ngen, and ngen-cal. Just install Docker Desktop and pull the image.

1.1 Install Docker Desktop

Windows:

- Go to https://www.docker.com/products/docker-desktop/ and download Docker Desktop for Windows.
- Run the installer and follow the prompts.
- When asked, enable WSL 2 (Windows Subsystem for Linux) and click Yes.
- Restart your computer when prompted.
- Launch Docker Desktop from the Start Menu.
- Wait until you see the green whale icon in the taskbar saying, "Docker Desktop is running."

Mac:

- Download Docker Desktop for Mac from the same link above.
- Choose the correct version: Apple Silicon (M1/M2/M3) or Intel.
- Drag Docker to your Applications folder and launch it.

1.2 Pull the Workshop Image

Open a terminal (Windows: search "Command Prompt"; Mac: search "Terminal") and run:

```
docker pull mdsaiduzzaman/nextgen-troute-workshop:2026-07-15
```

This downloads the workshop environment (~2 GB). It may take 10–20 minutes, depending on your connection speed.

1.3 Verify the Image

After the download finishes, confirm the image is on your computer:

```
docker images | grep nextgen-troute-workshop
```

You should see output like this:

```
nextgen-troute-workshop 2026-07-15 abc123 ... 2.58GB
```

If you see that line, you are ready for the workshop.

1.4 Create Your Data Folder

Windows (in Command Prompt):

```
mkdir %USERPROFILE%\workshop_data
```

Mac/Linux (in Terminal):

```
mkdir -p ~/workshop
```

1.5 Launch of the Workshop Container

Windows:

```
docker run --rm -it --shm-size=4g -p 8888:8888 -v %USERPROFILE%\workshop:/workspace/data mdsaiduzzaman/nextgen-troute-workshop:2026-07-15
```

Mac/Linux:

```
docker run --rm -it --shm-size=4g -p 8888:8888 -v ~/workshop:/workspace/data mdsaiduzzaman/nextgen-troute-workshop:2026-07-15
```

You will see a prompt like this inside the container:

```
root@workshop:/workspace#
```

Note: *Leave this terminal open for the entire workshop. Your work is saved in workshop_data on your computer.*

1.6 Launch JupyterLab

Inside the container, run:

```
jupyter lab --ip=0.0.0.0 --port=8888 --no-browser --allow-root --NotebookApp.token=''
```

Then open your browser and go to:

```
http://localhost:8888
```

You should see the JupyterLab interface. Open the notebooks folder to find the workshop notebooks.

Part 2: Build From Source (Linux)

Prerequisites & Environment Setup

Create a base working directory for all source code:

```
mkdir /path/to/your/directory/workshop
cd workshop
mkdir source_code
cd source_code/
```

Create and activate a Conda environment with Python 3.10:

```
conda create -n workshop python=3.10
```

```
conda activate workshop
```

All subsequent build steps are performed with the workshop environment active.

1 Compile T-Route

1.1 Clone the repository and install requirements

```
git clone https://github.com/said-uzzaman/t-route-MST.git
cd /Workshop/source_code/t-route-MST/
pip install -r requirements.txt
```

1.2 Run the compiler

```
./compiler.sh
```

Expected output:

```
using F90=gfortran
using CC=gcc
using NETCDFINC=/usr/include/openmpi-x86_64/
...
gfortran -g -c -O2 -fPIC ... -o diffusive.o diffusive.f90
diffusive.f90:1489:20:
 1489 |        pause !test
      |              1
Warning: Deleted feature: PAUSE statement at (1)
...
cp *.o ../../../src/troute-routing/troute/routing/fast_reach
```

Here, compiler.sh contains a conditional block for locating NetCDF including headers. If you face any issues while compiling, you must change the directory paths based on your local computer:

```
# if you have custom static library paths, uncomment below and export them
# export LIBRARY_PATH=<paths>:$LIBRARY_PATH
# if you have custom dynamic library paths, uncomment below and export them
# export LD_LIBRARY_PATHS=<paths>:$LD_LIBRARY_PATHS
if [ -z "$NETCDF" ]
then
  export NETCDFINC=/usr/include/openmpi-x86_64/
  # set alternative NETCDF variable include path, for example for WSL
  # (Windows Subsystems for Linux).
  #
  # EXAMPLE USAGE: export NETCDFALTERNATIVE=$HOME/.conda/envs/py39/include/
  # (before ./compiler.sh)
  if [ -n "$NETCDFALTERNATIVE" ]
  then
    echo "using alternative NETCDF inc ${NETCDFALTERNATIVE}"
    export NETCDFINC=$NETCDFALTERNATIVE
  fi
else
  export NETCDFINC="${NETCDF}"
fi
```

For example, if your NetCDF libraries and headers live under a custom local prefix, uncomment the LD_LIBRARY_PATHS line and point it at your library directory, and change NETCDFINC to your local include directory:

```
# if you have custom static library paths, uncomment below and export them
# export LIBRARY_PATH=<paths>:$LIBRARY_PATH
# if you have custom dynamic library paths, uncomment below and export them
export LD_LIBRARY_PATHS=/home/msmg8/local/lib/:$LD_LIBRARY_PATHS
if [ -z "$NETCDF" ]
then
  export NETCDFINC=/home/msmg8/local/include/
```

2 Build the Image for ngen

2.1 Clone NGIAB-CloudInfra

```
git clone https://github.com/said-uzzaman/NGIAB-CloudInfra.git
cd /home/msmg8/Workshop/source_code/NGIAB-CloudInfra/docker/
```

2.2 Edit the Dockerfile

```
nano Dockerfile
```

Set the base image and the repo/branch variables so the build pulls the correct forks of T-Route and ngen:

```
FROM rockylinux:9.1 AS base
ENV TROUTE_REPO=said-uzzaman/t-route-MST
ENV TROUTE_BRANCH=main
ENV NGEN_REPO=said-uzzaman/ngen
ENV NGEN_BRANCH=main
```

2.3 Build the image

```
docker buildx build -f Dockerfile -t awiciroh/ciroh-ngen-image .
docker images
```

This builds the ngen image tagged **awiciroh/ciroh-ngen-image**. Run docker images afterward to confirm the image was created.

3 Build the Image for ngen-cal

3.1 Clone ngen-cal (ngiab_cal branch)

```
git clone --branch ngiab_cal https://github.com/CIROH-UA/ngen-cal.git
cd /home/msmg8/Workshop/source_code/ngen-cal/
```

3.2 Build the image

```
docker build -t ngiab-cal:custom .
docker images
```

This builds the calibration image tagged **ngiab-cal:custom**. Run docker images afterward to confirm.